

Internet of Things

Lab 5



Project Overview: Wireless Voice Control



Core Objective

Implement a functional, wireless voice-controlled LED system leveraging the power of the ESP32 microcontroller.



Key Integration Points

Integration of advanced speech recognition (Google API), robust network protocols (WiFi), and embedded control.



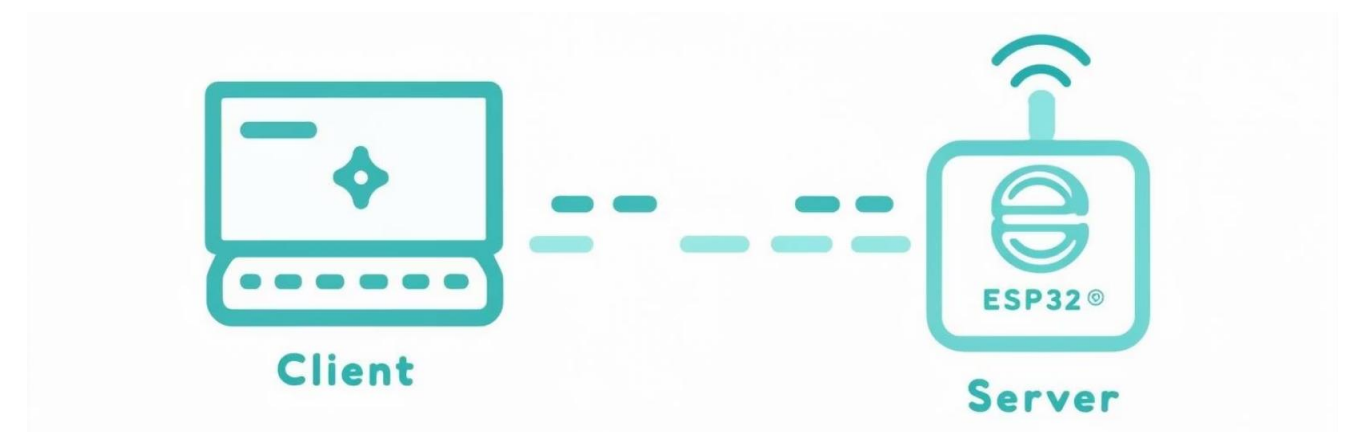
Demonstration Commands

Control the LED state using simple, predefined voice commands: "**ON**", "**OFF**", and "**SHUFFLE**" (1Hz blink).

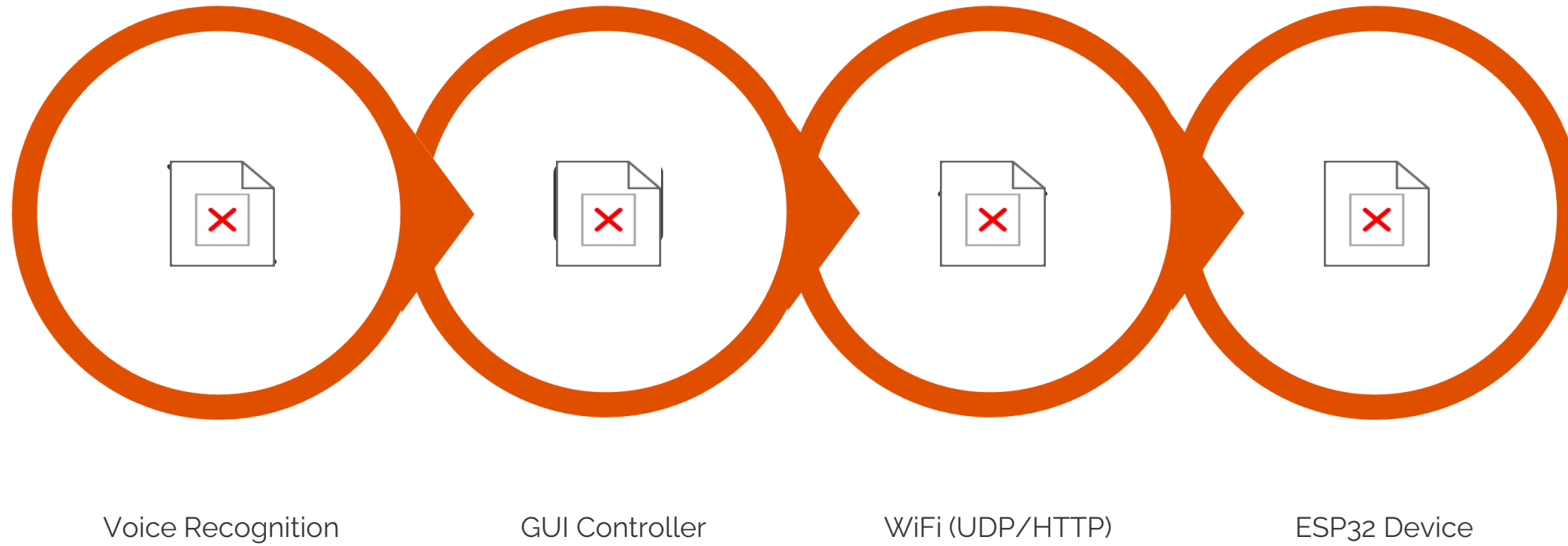
Distributed System

- **Cross-Platform Communication:** Seamless data transfer between a high-resource PC and a low-resource microcontroller.
- **Real-time Constraints:** Processing complex audio data quickly enough for perceived real-time response.
- **Network Reliability:** Ensuring robust command transmission and device discovery.

Client-Server Architecture



System Architecture: Client-Server Flow



Key Components in the Flow

- **Desktop Client:** Handles Audio Capture, Google API Recognition, and the Tkinter GUI.
- **Network Layer:** Uses UDP for initial device discovery and HTTP POST for reliable command transmission.
- **ESP32 Server:** Runs a MicroPython HTTP server to receive commands and manage GPIO control (Pin 32) for the LED.

Speech Recognition: From Sound Waves to Text

1

Audio Capture

PyAudio captures raw microphone input, typically at a 16kHz sampling rate, ready for processing.

2

Feature Extraction (MFCC)

The audio data is converted into Mel-Frequency Cepstral Coefficients (MFCC), which represent the spectral characteristics of the human voice.

3

Cloud Processing

The features are sent to the high-performance Google Speech Recognition API, which uses deep neural networks for accurate transcription.

4

Text Output

The result is a transcribed text string (e.g., "turn on"), which the client application then parses into a command.

```
text = recognizer.recognize_google(audio, language="en-US")
```

Cloud-based APIs offer superior accuracy and multi-language support compared to local, embedded recognition.

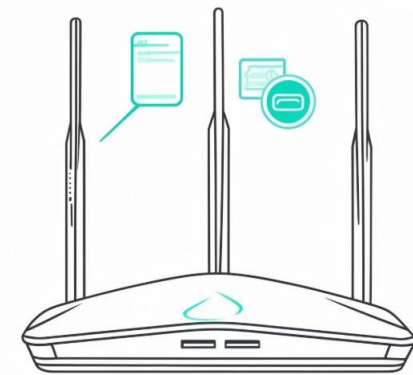
Network Protocols for IoT Reliability

UDP Broadcasting: Automatic Discovery

To eliminate manual IP configuration, the ESP32 broadcasts its address on the local network (255.255.255.255:12346).

📄 **Payload Example:** "ESP32_VOICE_LAB:192.168.1.9"

The Python client listens for this specific broadcast, extracts the IP, and establishes the communication path.



HTTP POST: Command Transmission

- Uses standard **HTTP/1.1 POST** requests.
- The transcribed command is sent as a simple, stateless payload (e.g., `command=turn_on`).
- The ESP32 server confirms receipt with a status message response.

This RESTful approach is reliable, standard, and easy to debug.

ESP32 & Python

ESP32 Hardware Core

- **Connectivity:** Integrated WiFi 802.11 b/g/n
- **Control:** GPIO used for digital output (3.3V / 0V) to control the LED circuit.

MicroPython for Rapid Development

MicroPython is an optimized Python 3 implementation that runs directly on microcontrollers, allowing for fast iteration and leveraging Python's rich ecosystem.

1 Lightweight

Designed for resource-constrained environments (16KB+ RAM).

2 Real-Time

Supports time-critical tasks and hardware interaction.

3 Simplicity

Enables direct GPIO and timer control using high-level Python syntax.

GPIO & Timer Control (Shuffle Mode)

The "SHUFFLE" command utilizes the internal timer to achieve precise 1Hz blinking.

```
timer = Timer(0)timer.init(period=1000, mode=Timer.PERIODIC, callback=toggle_led)
```

Modules

1. ESP32 Server (`esp32_led_server.py`)

The embedded application running the HTTP server on Port 80. It directly translates received commands (`turn_on`, `shuffle`) into physical actions via GPIO.

2. Communication Module (`esp32_controller.py`)

Handles the critical networking layer: automatic IP discovery (UDP socket) and reliable command delivery (HTTP requests library). Provides an essential abstraction layer.

3. GUI Application (`gui_control.py`)

A multi-threaded Tkinter application providing both a voice interface and a debug mode with manual test buttons, ensuring a non-blocking user experience.

Technical Detail: Timer-Based PWM

```
def shuffle_callback(timer):    led.value(not led.value()) # Toggle state
```

This code produces a highly reliable 50% duty cycle, 1Hz square wave for the blinking effect, demonstrating embedded real-time capability.

Metrics

~95%

Accuracy

Voice recognition accuracy under clear speech conditions.

<200ms

Response Time

Command latency (end-to-end), critical for user experience.

1.000Hz

Shuffle Frequency

Measured accuracy of the timer-based PWM control ($\pm 0.005\text{Hz}$).

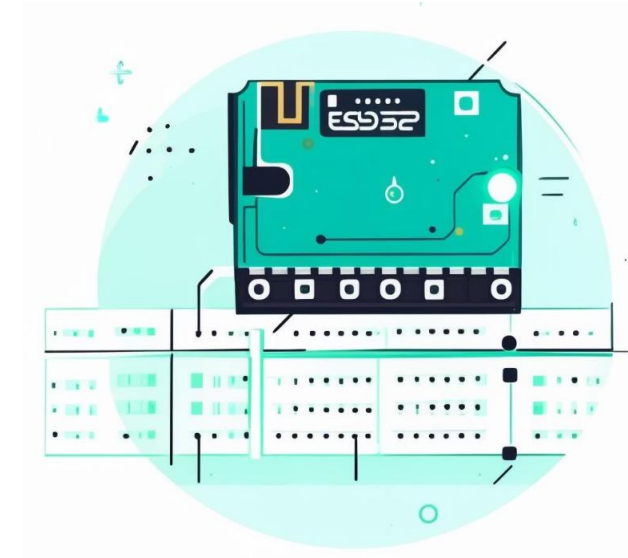
Voice Commands	Successfully executed all 'ON', 'OFF', and 'SHUFFLE' requests in over 100 tests.
Network Reliability	Handled rapid command sequences and successfully recovered from simulated network failure.
Discovery Time	ESP32 IP address discovered automatically within 2–5 seconds on network connection.

Conclusion

Future Work: Scaling the System

-  **Multi-Device Control:** Adapt the discovery and command structure to manage multiple ESP32 units concurrently.
-  **Custom Wake Word:** Implement local, low-power speech processing for a custom trigger (e.g., "Hey ESP32").
-  **MQTT Protocol:** Migrate from HTTP to MQTT for better scalability, message queuing, and reduced overhead.
-  **Variable Control:** Integrate PWM for analog-like control, enabling variable LED brightness or color changes.

Project Conclusion



This project successfully validates a robust and practical method for integrating cloud-based speech recognition with embedded systems using MicroPython and the ESP32 platform.

 **Technologies Used:** MicroPython, SpeechRecognition, UDP, HTTP, GPIO Control.